MTH 451/551
Lecture 3

Prof. W. Pazner

1 Floating Point Numbers and Arithmetic

[Textbook reference: §1.4]

As discussed in the first lecture, the point of numerical analysis is **to use computer algorithms to solve mathematics problems**. Therefore, it is supremely important to understand how numbers are represented by the computer.

1.1 Prelude: Integer Representations

For integers, there is no real ambiguity. Integers are represented by the computer using their representation in binary. Typically, integers are stored using a fixed number of bits (for example, 32 or 64), which allows us to represent all integers in a finite range. A **bit** is simply a placeholder that can take the value 0 or 1. Representing integers beyond that range (i.e. with larger magnitude) simply requires using more bits. As an example, if we are encoding non-negative integers, we can represent the numbers 0, 1, 2, ..., 7 using three bits. This is a total of $8 = 2^3$ representable numbers, corresponding to ways to choose from $\{0, 1\}$ three times.

Number	Binary Representation
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

The number associated with the binary representation $b_2 b_1 b_0$ is $\sum_{k=0}^{2} b_k 2^k$. If we wished to represent *signed* numbers (positive or negative), we would use one bit of information to encode the

sign (there are various ways of doing this). The upshot of this discussion is that it is relatively straightforward and uncontroversial how to represent integers on the computer.

1.2 Floating Point Numbers

As soon as we want to represent *real numbers* (or even just *rational numbers*), the situation becomes more complicated. As opposed to the integer case, where a given number of bits can represent all possible integers within a range, the same is fundamentally impossible for real numbers, because every interval contains infinitely many of them, and so they cannot all be representable with finite information. This means that, even when representing numbers within a given range, we need to consider issues of precision and rounding. **Floating point numbers** are the most commonly (almost universally) used method to represent these kinds of numbers on the computer.

Before we define floating point numbers, it is useful to contrast them with the conceptually simpler but generally less useful *fixed point numbers*. We can think of fixed point numbers as "fixing the decimal point" in an integer number, allowing us to represent fractional numbers. Essentially, this is the same as fixing a denominator d, and then just representing integers, which get multiplied by $1/d$. These types of numbers are often used in financial calculations, where quantities in dollars are represented using a denominator of $d = 100$, allowing the exact representation of all dollars-and-cents quantities within a range. However, outside of this and some other niche applications, fixed point numbers are relatively rarely used, in favor of the much more flexible *floating point numbers*. The key advantage of floating point numbers is that, essentially, the fractional multiplier is also encoded as part of the number. This way, rather than "fixing" the position of the decimal point, it is allowed to "float".

Floating point numbers divide up the bits to represent three parts of the number: the sign, the mantissa, and the exponent. These terms correspond to representing a number as

$$x = (-1)^s \times m \times b^e,$$

where s encodes the **sign** (either 0 or 1), $m \in [1, b)$ is the **mantissa**, and e is the **exponent**; here, b is called the **base** of the representation. This is essentially the same as "scientific notation" for numbers, which you may be familiar with. So, to represent a floating point number, we need to encode s, m, and e. On the computer, it is natural to use **binary** (base two) representation, and we take $b = 2$.

The sign s is represented using a single bit. The number of bits used to represent the mantissa and exponent depends on the exact choice of representation. Luckily, these representations have been standardized in a standard called IEEE 754 that is basically universal among all computers. This standard defines how to represent floating point numbers with 32, 64, or 128 bits.

Total Bits	Sign	Mantissa	Exponent
32	1	23	8
64	1	52	11
128	1	112	15

32-bit floating point numbers are called **single precision**. Likewise, 64-bit floating point numbers are called **double precision**, and 128-bit floating point numbers are called **quadruple precision**. In numerical analysis, double precision is the most widely used. In other fields, such as machine learning, single precision (or even half precision, using 16 bits) is more widely used; similarly for computer graphics. In this course, we will focus on **64-bit double-precision floating point numbers**.

The exponent e is represented by 11 bits, so it can take on a total of $2^{11} = 2{,}048$ possible values; these give the range $-1022 \leq e \leq 1023$, along with two reserved values used for special numbers. The 52-bit mantissa $m \in [1, 2)$ is given by

$$m = 1 + \sum_{k=1}^{52} d_k 2^{-k} = (1.d_1 d_2 \cdots d_{52})_2,$$

where d_k is the value of the kth mantissa bit. The subscript "2" in the expression $(1.d_1 d_2 \cdots d_{52})_2$ indicates that the digits after the "point" are interpreted in base 2. The sign bit s can be either 0 or 1. The number represented by these 64 bits is

$$y = (-1)^s \times m \times 2^e = \pm(1.d_1 d_2 \cdots d_{52})_2 \times 2^e.$$

We use the symbol $\mathbb{F}$ to denote the (finite) set of all such 64-bit numbers, together with the number zero. *(Note: these are the **normalized** floating-point numbers. Some special numbers, such as positive infinity and negative infinity, as well as so-called "subnormal" numbers, can also be represented in this format, but these are beyond the current scope).*

The mantissa is always in the interval $[1, b)$. We can be more precise. The smallest mantissa is when all its bits are zero, and is exactly equal to 1. The largest mantissa is when all its bits are one, and is equal to

$$1 + \sum_{k=1}^{52} 2^{-k} = \sum_{k=0}^{52} (2^{-1})^k = \frac{1 - 2^{-53}}{1 - 2^{-1}} = 2 - 2^{-52}.$$

The spacing between successive mantissa values, which is called **machine epsilon**, is

$$\epsilon = 2^{-52} = 2.220446049250313080847263336181640625 \times 10^{-16}.$$

The spacing between consecutive floating point numbers in $[2^e, 2^{e+1}]$ is $\epsilon \times 2^e$ (and likewise for $[-2^{e+1}, -2^e]$). For this reason, and as the next theorem will demonstrate, it is often said that double precision has (approximately) **16 (decimal) digits of precision**.

Another special value in $\mathbb{F}$ is called **unit roundoff**, and is given by

$$u = \frac{1}{2}\epsilon = 2^{-53} = 1.110223024625156540423631668090820312 5 \times 10^{-16}.$$

This number features in error analysis of rounding and floating point arithmetic. The smallest and largest positive numbers in $\mathbb{F}$ are

$$F_{\min} = 2^{-1022} \approx 2.2250738585072013830902327173324040642 \times 10^{-308},$$
$$F_{\max} = (1-u)2^{1024} \approx 1.7976931348623157081452742373170435680 \times 10^{308}.$$

We say that a real number $x \in \mathbb{R}$ is **in the range of** $\mathbb{F}$ if $F_{\min} \leq |x| \leq F_{\max}$ or if $x = 0$. For such an x, we define the **rounded value** of x to be the number $\mathrm{fl}(x) \in \mathbb{F}$ nearest to x. In case of a tie, we round to the nearest element of $\mathbb{F}$ with $d_{52} = 0$ (we say that **ties round to even**; this is an unbiased rounding method).

The following theorem bounds the relative error between a number x in the range of $\mathbb{F}$ and its rounded value $\mathrm{fl}(x)$, in terms of the unit roundoff u.

Theorem 1. *If $x \in \mathbb{R}$ is in the range of $\mathbb{F}$, then $\mathrm{fl}(x) = (1+\delta)x$ for some $|\delta| < u$. In other words, for non-zero x, the relative error in rounding is smaller than u,*

$$\frac{|x - \mathrm{fl}(x)|}{|x|} = |\delta| < u.$$

Proof. Without loss of generality, we may assume $x \neq 0$ since if $x = 0$ then $\mathrm{fl}(x) = x$ and we may take $\delta = 0$. The first step is to write x as

$$x = \pm m \times 2^e,$$

where $m \in [1, 2)$ is the mantissa and $e \in \mathbb{Z}$ is the exponent. All nonzero real numbers can be written this way. If $m = 1$, then again $\mathrm{fl}(x) = x$ and $\delta = 0$, so we now consider only the case that $m > 1$. As remarked above, the floating point numbers between 2^e and 2^{e+1} are evenly spaced, with spacing $\epsilon \times 2^e$ (and the same for the floating point numbers between -2^e and -2^{e+1}). So, x lies in some interval $[a, a + \epsilon 2^e]$, where both endpoints are in $\mathbb{F}$. The closer of the two endpoints is $\mathrm{fl}(x)$ (and if there is a tie, apply the round-to-even rule). The distance from x to the closest of the endpoints is at most half the interval width, i.e. $\frac{1}{2}\epsilon 2^e = u2^e$. Consequently, $|x - \mathrm{fl}(x)| \leq u2^e$. Since $|x| > 2^e$, it follows that

$$\frac{|x - \mathrm{fl}(x)|}{|x|} \leq \frac{u2^e}{|x|} < \frac{u2^e}{2^e} = u.$$

Let $\delta = (\mathrm{fl}(x) - x)/x$. It can be seen immediately that $\mathrm{fl}(x) = (1+\delta)x$. The above inequality implies that $|\delta| < u$. □

Note that even some simple rational numbers such as $4/3$ are not in $\mathbb{F}$. To see this, recall that for $0 < r < 1$, we can compute the limit of the geometric series,

$$\sum_{k=0}^{\infty} r^k = \frac{1}{1-r}.$$

Let $r = 2^{-m}$ for natural number m. Then,

$$\frac{1}{1-2^{-m}} = \sum_{k=0}^{\infty} 2^{-km} = (1.\underbrace{0\ldots01}_{m}\underbrace{0\ldots01}_{m}\cdots)_2.$$

This is an infinitely repeating pattern of digits after the point. If $m \geq 2$, then the digits are not all 1. Since every number in $\mathbb{F}$ has a finite binary expansion, this implies that $\frac{1}{1-2^{-m}} \notin \mathbb{F}$. As an example,

$$\frac{4}{3} = \frac{1}{1-2^{-2}} = (\mathbf{1.}\underbrace{\mathbf{0101\cdots01}}_{52}01\cdots)_2 = \left(\sum_{k=0}^{26} 2^{-2k}\right) + \sum_{k=27}^{\infty} 2^{-2k}.$$

The bolded number (which is equal to the sum in parentheses) is the largest number in $\mathbb{F}$ that is less than $4/3$. The closest two floating point numbers above and below $4/3$ are

$$a = (1.0101\cdots0101)_2 = \sum_{k=0}^{26} 2^{-2k},$$
$$b = (1.0101\cdots0110)_2 = \sum_{k=0}^{25} 2^{-2k} + 2^{-51} = a + 2^{-52}.$$

We can see that

$$|4/3 - a| = (0.\underbrace{\mathbf{00\cdots00}}_{52}0101\cdots)_2,$$
$$|4/3 - b| = (0.\underbrace{\mathbf{00\cdots00}}_{52}1010\cdots)_2,$$

and so a is twice as close to $4/3$ as b is. Therefore,

$$\begin{aligned}\mathrm{fl}(4/3) = a &= (1.0101\cdots0101)_2 \\ &= \underbrace{\mathbf{1.333333333333333}}_{\text{16 digits}}2593184650249895639717578887939453125.\end{aligned}$$

Note that every number in $\mathbb{F}$ has an *exact* decimal representation, but it may take many decimal places to write it out in full. In general, only the first ~16 of those digits will be meaningful.

1.3 Floating Point Arithmetic

We have now defined which numbers we can represent on the computer: those in $\mathbb{F}$ (as well as some "special" numbers mentioned above, which we will not discuss further). We now need to define how to perform arithmetic on these numbers. The four fundamental arithmetic operations are $+, -, \times, \div$. In this section, we will use $\circ$ to generically denote one of these operations, i.e. to serve as a stand-in for any of them.

Even if $x, y \in \mathbb{F}$, it may be that $x \circ y \notin \mathbb{F}$. This is clear for division. Even though $3 \in \mathbb{F}$ and $4 \in \mathbb{F}$, we saw earlier that $4 \div 3 \notin \mathbb{F}$. It is also true for the other three operations. For example, $1 \in \mathbb{F}$ and $2^{-53} \in \mathbb{F}$, but $1 + 2^{-53} \notin \mathbb{F}$.

Let $+_{\text{fl}}, -_{\text{fl}}, \times_{\text{fl}}, \div_{\text{fl}}$ denote the *floating point* versions of each of the operations, denoted generically by $\circ_{\text{fl}}$. These are functions $\circ_{\text{fl}} : \mathbb{F} \times \mathbb{F} \to \mathbb{F}$. (Division by zero requires a special definition; we avoid discussing this case). For $x, y \in \mathbb{F}$, we define

$$x \circ_{\text{fl}} y = \text{fl}(x \circ y) \in \mathbb{F}.$$

This means that the computed value $x \circ_{\text{fl}} y$ is given by rounding the exact value $x \circ y$ to the nearest floating point number. This same property is required or suggested for other operations, such as $\sqrt{x}$. Function such as $\exp, \sin, \cos, \log, \tan$, etc. are **not** required to give the rounded version of the exact value. This will be evident in some of the examples we consider.

While some familiar properties of standard arithmetic operations still hold for their floating point equivalents, other properties *do not* hold. For example, addition and multiplication are commutative in both exact arithmetic and in floating point arithmetic:

$$x +_{\text{fl}} y = \text{fl}(x + y) = \text{fl}(y + x) = y +_{\text{fl}} x,$$
$$x \times_{\text{fl}} y = \text{fl}(x \times y) = \text{fl}(y \times x) = y \times_{\text{fl}} x.$$

However, floating point operations are *not* generally associative. To illustrate this, consider (in exact arithmetic),

$$x = 1 + \epsilon = 1 + 2u = (1 + u) + u = 1 + (u + u).$$

However, in floating point arithmetic, note that

$$1 +_{\text{fl}} u = 1, \qquad \text{and} \qquad u +_{\text{fl}} u = \epsilon,$$

and so

$$(1 +_{\text{fl}} u) +_{\text{fl}} u = 1 +_{\text{fl}} u = 1 \neq x,$$

but

$$1 +_{\text{fl}} (u +_{\text{fl}} u) = 1 +_{\text{fl}} \epsilon = 1 + \epsilon = x.$$

Note that $\text{fl}(x)$ is expected to have ~16 correct digits, and so the basic arithmetic operations are expected to be computed with a high degree of accuracy, since $x +_{\text{fl}} y = \text{fl}(x + y)$. However, these small errors can accumulate, sometimes in quite significant ways, leading to calculations that have much larger relative errors. This is often the result of subtracting two distinct but very close numbers.

Example 1. Let $x = \exp(2^{-n})$. For large n, the exponent is very close to zero, and so x is very close to one. Let $y = 1$. We compute $\text{fl}(x) -_{\text{fl}} y$ for various values of n, and show the result in the table below.

n	$\mathrm{fl}(x)$	$\mathrm{fl}(x) -_{\mathrm{fl}} y$	$\mathrm{fl}(x-y)$
25	$1 + 2^{-25} + 2^{-51}$	$2^{-25} + 2^{-51}$	$2^{-25} + 2^{-51} + 2^{-77}$
26	$1 + 2^{-26} + 2^{-52}$	$2^{-26} + 2^{-52}$	$2^{-26} + 2^{-53}$
27	$1 + 2^{-27}$	2^{-27}	$2^{-27} + 2^{-55}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
51	$1 + 2^{-51}$	2^{-51}	$2^{-51} + 2^{-103}$
52	$1 + 2^{-52}$	2^{-52}	$2^{-52} + 2^{-104}$
53	$1 + 2^{-52}$	2^{-52}	2^{-53}
54	1	0	2^{-54}
$\vdots$	$\vdots$	$\vdots$	$\vdots$

What we see from this table is that for $n \geq 54$,

$$\tilde{z} = \mathrm{fl}(x) -_{\mathrm{fl}} y = 0, \qquad \text{but} \qquad z = x - y > 0.$$

This means that the *relative error* is

$$\frac{|z - \tilde{z}|}{|z|} = \frac{|z|}{|z|} = 100\%.$$

For this calculation, there are no correct significant digits. ♦

The previous example may seem somewhat contrived. The next example shows that in practical situations, being aware of potential round-off errors can be extremely important.

Example 2. Recall the forward difference approximation to the derivative,

$$f'(c) = \frac{f(c+h) - f(c)}{h} + \mathcal{O}(h).$$

Take $f(x) = \exp(x)$ and $c = 0$. We know that $f'(x) = f(x) = \exp(x)$, and so $f'(0) = 1$. We compute the difference quotient in floating point arithmetic. We expect that the error will go to zero linearly as $h \to 0$. However, for very small values of h, the error from subtractive cancellation can be catastrophic. From the table in the previous example, we see that for $n \geq 54$, $\mathrm{fl}(\exp(2^{-n})) = 1$. Therefore, for $h = 2^{-n}$,

$$\frac{\mathrm{fl}(\exp(h)) -_{\mathrm{fl}} \exp(0)}{h} = \frac{1 -_{\mathrm{fl}} 1}{h} = 0, \qquad (n \geq 54),$$

and the relative error is again 100%. We can also see from the previous table that for $n = 53$, the relative error is 100%. In this case, the numerator is 2^{-52} and the denominator is 2^{-53}, and so the difference quotient gives an approximation of 2, but the exact answer is 1. The table below shows the finite-difference approximation and relative error for values of $h = 2^{-n}$.

n	$(\mathrm{fl}(\exp(h)) -_{\mathrm{fl}} 1) \div_{\mathrm{fl}} h$	Relative Error
22	$1 + 2^{-23}$	$2^{-23} \approx 1.19 \times 10^{-7}$
23	$1 + 2^{-24}$	$2^{-24} \approx 5.96 \times 10^{-8}$
24	$1 + 2^{-25}$	$2^{-25} \approx 2.98 \times 10^{-8}$
25	$1 + 2^{-26}$	$2^{-26} \approx 1.49 \times 10^{-8}$
26	$1 + 2^{-26}$	$2^{-26} \approx 1.49 \times 10^{-8}$
27	1	0
⋮	⋮	⋮
52	1	0
53	2	1
54	0	1

The upshot is that, while the finite difference error decreases as $h \to 0$, we cannot take h too small, otherwise round-off error will destroy the accuracy.

Note that the exact values obtained in the example above (where the relative error is zero) are essentially coincidences because of the particular choice of f and c. If we choose $c = 1$, then we obtain the following results.

n	$(\mathrm{fl}(\exp(1 +_{\mathrm{fl}} h)) -_{\mathrm{fl}} \mathrm{fl}(e)) \div_{\mathrm{fl}} h$	Relative Error
16	2.7183025674	7.63×10^{-6}
18	2.7182870132	1.91×10^{-6}
20	2.7182831247	4.77×10^{-7}
22	2.7182821538	1.20×10^{-7}
24	2.7182819098	2.99×10^{-8}
26	2.7182818651	1.35×10^{-8}
28	2.7182818651	1.35×10^{-8}
30	2.7182822227	1.45×10^{-7}
32	2.7182826996	3.20×10^{-7}
34	2.7182846069	1.02×10^{-6}
36	2.7182922363	3.83×10^{-6}
38	2.7182617188	7.40×10^{-6}
40	2.7182617188	7.40×10^{-6}
⋮	⋮	⋮
50	3	1.04×10^{-1}
51	3	1.04×10^{-1}
52	4	4.72×10^{-1}
53	0	1

This example shows the most common behavior of the error for finite difference quotients: at first, the error decreases with h, since the $\mathcal{O}(h)$ term gets smaller. Then, for sufficiently small h, the round-off error becomes, significant, and the error starts to increase until it completely dominates. Notice that the error achieved a minimum of about 10^{-8}, which is very far from the

16 decimal digits of accuracy we would hope to obtain. We can help address this issue by using a centered difference approximation, which has error $\mathcal{O}(h^2)$.

n	$(\mathrm{fl}(\exp(1 +_{\mathrm{fl}} h)) -_{\mathrm{fl}} \mathrm{fl}(\exp(1 -_{\mathrm{fl}} h))) \div_{\mathrm{fl}} (2h)$	Relative Error
8	2.7182887414125	2.54×10^{-6}
10	2.7182822605182	1.59×10^{-7}
12	2.7182818554629	9.93×10^{-9}
14	2.7182818301480	6.21×10^{-10}
16	2.7182818285655	3.92×10^{-11}
18	2.7182818284491	3.66×10^{-12}
20	2.7182818283327	4.65×10^{-11}
22	2.7182818287984	1.25×10^{-10}
24	2.7182818278670	2.18×10^{-10}
26	2.7182818353176	2.52×10^{-9}
28	2.7182818055153	8.44×10^{-9}
30	2.7182819843292	5.73×10^{-8}
32	2.7182817459106	3.04×10^{-8}
34	2.7182807922363	3.81×10^{-7}
⋮	⋮	⋮
50	2.75	1.17×10^{-2}
51	2.5	8.03×10^{-2}
52	3	1.04×10^{-1}
53	0	1

Here, we see that the error achieves a minimum of about 10^{-12}, which is 10,000 times smaller than in the case of forward differences.

We can predict roughly what error is achievable by considering both sources of error: the truncation error from the Taylor expansion, and round-off error from the floating point arithmetic. The truncation error scales like h^2. The round-off error in the numerator will scale like $2ue$, and so after dividing by the denominator $2h$ (and taking the relative error, with exact value $f'(1) = e$), we see that the contribution to the relative error from round-off scales like u/h. So, the overall error will scale like $\boxed{\eta(h) = h^2 + u/h}$.

We can find the h that minimizes $\eta(h)$. To do this, we find the critical point $\eta'(h) = 2h - uh^{-2} = 0$. This gives $h = \sqrt[3]{u/2} = \sqrt[3]{2^{-54}} = 2^{-18}$. This model predicts that the error will be minimized at $\boxed{n = 18}$ and the error will be bounded roughly $2^{-36} + 2^{-53}/2^{-18} = 3 \times 2^{-36} \approx 4 \times 10^{-11}$. Indeed, in our table, the minimum is achieved at $n = 18$, and the observed error is less than the bound provided by the model. ♦

Example 3. For functions which are *analytic* (complex differentiable around x), and which take real values on the real line, then we can use complex variable methods to get even more accurate approximations to derivatives. Counterintuitively, these methods use *complex numbers* to

approximate results that are always real numbers. If we evaluate the Taylor series of f about x at the point $x + \mathrm{i}h$ (where i is the imaginary unit), we obtain

$$f(x + \mathrm{i}h) = f(x) + \mathrm{i}hf'(x) - \frac{h^2}{2}f''(x) - \frac{\mathrm{i}h^3}{3!}f^{(3)}(x) + \cdots,$$

and so

$$\frac{\mathrm{im}(f(x + \mathrm{i}h))}{h} = f'(x) + \mathcal{O}(h^2).$$

This is a second-order approximation requiring only one (complex) evaluation of f. Furthermore, **there is no subtraction** in the numerator, and since the imaginary part of $x + \mathrm{i}h$ is $\mathcal{O}(h)$, the round-off error in computing $\mathrm{fl}(f(x + \mathrm{i}h))$ will scale like uh. Therefore, we can model the error as $h^2 + uh/h = h^2 + u$. The round-off term *does not grow* as $h \to 0$, allowing us to use extremely small values of h and still obtain accurate results.

n	$\mathrm{fl}(\mathrm{im}(\exp(1 + \mathrm{i}h))) \div_{\mathrm{fl}} h$	Relative Error
8	2.718274915516147	2.54×10^{-6}
10	2.718281396399805	1.59×10^{-7}
12	2.718281801455341	9.93×10^{-9}
14	2.718281826771314	6.21×10^{-10}
16	2.718281828353562	3.88×10^{-11}
18	2.718281828452453	2.43×10^{-12}
20	2.718281828458633	1.52×10^{-13}
22	2.718281828459019	9.53×10^{-15}
24	2.718281828459044	5.43×10^{-16}
26	2.718281828459045	5.32×10^{-17}
28	2.718281828459045	5.32×10^{-17}
30	2.718281828459045	5.32×10^{-17}
32	2.718281828459045	5.32×10^{-17}
34	2.718281828459045	5.32×10^{-17}
⋮	⋮	⋮
200	2.718281828459045	5.32×10^{-17}
201	2.718281828459045	5.32×10^{-17}
202	2.718281828459045	5.32×10^{-17}
203	2.718281828459045	5.32×10^{-17}

From these results, we see that the complex step method achieves as good accuracy as is possible, without needed to balance the competing forces of truncation error and round-off error. ⧫

Our final example concerns linear algebra.

Example 4. For $a \neq 0$, consider the solution of the following linear system by Gaussian elimination with back-substitution,

$$\begin{pmatrix} a & 1 \\ 1 & 1 \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \longrightarrow \quad \begin{pmatrix} a & 1 \\ 0 & 1 - 1/a \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 2 - 1/a \end{pmatrix},$$

which yields

$$y = \frac{2 - (1/a)}{1 - (1/a)},$$
$$x = \frac{1 - y}{a}.$$

For $a = 2^{-54}$, both x and y are very nearly 1. However, if we evaluate these formulas in floating point arithmetic, we obtain $y = 1 - 2^{-53}$ (which is in fact $\mathrm{fl}(y)$, the best possible value), which then gives $x = 2^{-53}/2^{-54} = 2$. The relative error in x is therefore $1 - 2^{-53}$, i.e. essentially 100%.

This example illustrates that simple linear algebra problems where *all entries of the matrices and vectors* can be represented *exactly* in floating point (and the solution can be represented *very accurately* in floating point) can still lead to catastrophic round-off errors.

However, through careful design of linear algebra algorithms, these problems can be avoided. Such algorithms are called **numerically stable** methods. Indeed, if we use a standard Gaussian elimination routine on the computer to solve this problem, we will obtain answers that are extremely accurate. The methods used are specifically designed to avoid these bad round-off errors. ♦